Retrospectiva del Proyecto Inicial: Slot Machine

Ciclos 1 a 4 y Cierre (Ciclo 5)

Samuel Mena Serrato, Wilson Morales Sánchez

Octubre 2026

Resumen del estado del proyecto

Ciclo	Mini-ciclos	Estado	Pruebas
1	1.1 a 1.5	Terminado	`SlotMachineC1Test` (12)
2	2.1 a 2.5	Terminado	`SlotMachineC2Test` (30) y `SlotMachineCC2Test` (3)
3	3.1 a 3.5	Terminado	`SlotMachineContestTest` (7), `SlotMachineContestCTest` (2) y `SlotMachineC3Test` (7)
4	4.1 a 4.7	Terminado	`SlotMachineC4Test` (38) y `SlotMachineCC4Test` (3)
Cierre	5.1 a 5.5	Terminado	`SlotMachineAcceptanceTest` (2). **104 de 104 pruebas pasan.**

Tiempo (horas)	Ciclo 1	Ciclo 2	Ciclo 3	Ciclo 4	Cierre	Total
Wilson Daniel Morales Sánchez	11	11	12	14	11	**59**
Samuel Mena Serrato	11	11	12	14	11	**59**

Ciclo 1: simulador básico

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Mini-ciclo	Contenido	Justificación
1.1	Crear la máquina vacía e invisible: `SlotMachine()`	Es la base de todo lo demás.
1.2	Adicionar y eliminar ruedas y símbolos: `addWheel`, `delWheel`, `addSymbol`, `delSymbol`	Sin ruedas y símbolos no hay nada que girar.
1.3	Girar y consultar: `spin`, `symbols`, `configuration`	Es el comportamiento principal del simulador.
1.4	Jackpot: `isJackpot`	Depende de poder consultar la configuración.
1.5	Visibilidad, terminar y `ok()`: `makeVisible`, `makeInvisible`, `exit`	Se puede trabajar invisible y luego añadir el dibujo.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué? Terminado (1.1 a 1.5). Todos los requisitos funcionan en modo visible e invisible. En el Cierre se agregó `SlotMachineC1Test` para probarlos.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? Wilson Daniel Morales Sánchez: 11 horas. Samuel Mena Serrato: 11 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué? Tener desde el primer ciclo una máquina que se puede usar completa (crear, agregar, girar, consultar y ver), porque todos los ciclos siguientes se construyeron sobre ella.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo? Dibujar la máquina

con las figuras del curso (`Canvas`, `Rectangle`, `Circle`) y mantener el dibujo al día después de cada operación. Lo resolvimos con un método que vuelve a ubicar el fondo y las ruedas cada vez que algo cambia, y solo cuando la máquina está visible.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar? Bien: dividir los requisitos entre los dos y revisar juntos el resultado. A mejorar: escribir las pruebas al mismo tiempo que el código, no al final.

7. ¿Cuál fue la práctica XP más útil? ¿Por qué? Diseño simple: empezar por lo mínimo que funcionara y agregar el resto por partes.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Guía del Proyecto Inicial 2026-2 (Ciclo 1), enunciado del problema I Slot Machine (ICPC 2025) y el proyecto `shapes` visto en clase. La más útil fue la guía, porque define cada requisito.

Ciclo 2: nuevas operaciones

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Mini-ciclo	Contenido	Justificación
2.1	`swap`	Es independiente de las demás operaciones.
2.2	`lock` / `unlock`	Los giros siguientes deben respetar el bloqueo.
2.3	`spin(wheel)` y `spin(wheel, steps)`	Usan el bloqueo del mini-ciclo anterior.
2.4	`spin(String[])`, `placeSymbol` y `distinctSymbols`	Colocación directa de símbolos y consultas.
2.5	Pruebas `SlotMachineC2Test` y `SlotMachineCC2Test`	Para cerrar el ciclo con pruebas.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué? Terminado (2.1 a 2.5). Las 33 pruebas de C2 y CC2 siguen pasando en el Cierre.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? Wilson Daniel Morales Sánchez: 11 horas. Samuel Mena Serrato: 11 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué? Agregar todas las operaciones nuevas sin cambiar las del Ciclo 1, y dejarlas probadas con 33 pruebas de unidad y compartidas.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo? Que los giros respetaran las ruedas bloqueadas en todas las formas de `spin`. Lo resolvimos revisando el bloqueo en cada versión de `spin` antes de mover la rueda, y lo comprobamos con las pruebas de `lock`.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar? Bien: escribir las pruebas del ciclo y correrlas después de cada cambio. A mejorar: revisar que cada prueba compruebe lo que dice su nombre. En el Ciclo 4 encontramos que `distinctSymbolsShouldNotCountDuplicatesTwice` no agregaba ruedas, y la corregimos.

7. ¿Cuál fue la práctica XP más útil? ¿Por qué? Testing, porque las pruebas compartidas con otros grupos (CC2) mostraron casos que no habíamos pensado.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Guía del Proyecto Inicial 2026-2 (Ciclo 2) y materiales de clase sobre pruebas de unidad con JUnit. La más útil fue la guía.

Ciclo 3: resolver la maratón

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos. Primero nos enfocamos en entender el problema y revisar el diseño que ya teníamos. Después implementamos las nuevas funciones, y finalmente nos concentramos en las pruebas y en corregir los errores que fueron apareciendo. Así podíamos revisar cada etapa antes de seguir.

Mini-ciclo	Contenido	Justificación
3.1	Entender el problema y revisar el diseño que ya teníamos	Antes de programar había que saber qué pedía el problema I de la maratón.
3.2	`SlotMachine(n)` con ruedas en posiciones al azar	Es la máquina que hay que resolver.
3.3	`SlotMachineContest.solve(n)` y `solve(machine, n)` usando solo `spin` y `distinctSymbols`	Es la restricción del ciclo: resolver sin leer la configuración.
3.4	`simulate(n)`: la misma solución, pero visible	Para ver la solución funcionando.
3.5	Pruebas `SlotMachineContestTest` y `SlotMachineContestCTest`, y corrección de errores	Para comprobar la solución con distintos tamaños.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué? Terminado (3.1 a 3.5). `solve` lleva al jackpot una máquina ya creada. En el Cierre se agregó `SlotMachineC3Test`, que también lo prueba.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? Wilson Daniel Morales Sánchez: 12 horas. Samuel Mena Serrato: 12 horas. El tiempo se usó principalmente en programar, revisar el código, hacer pruebas y corregir.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué? Que la solución pudiera trabajar con una máquina ya creada y llevarla al jackpot. También agregamos las nuevas funciones sin eliminar lo que ya habíamos desarrollado.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo? Hacer que el algoritmo encontrara los movimientos para resolver la máquina sin acceder directamente a la configuración de sus ruedas. Lo resolvimos haciendo pruebas, revisando cómo funcionaban `spin()` y `distinctSymbols()` y ajustando el algoritmo hasta que funcionó.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar? Bien: dividirnos las tareas, ayudarnos cuando alguno tenía problemas y probar los cambios para no dañar lo que ya funcionaba. A mejorar: organizarnos mejor desde el principio y dejar más tiempo para las pruebas finales.

7. ¿Cuál fue la práctica XP más útil? ¿Por qué? Testing, porque nos permitió comprobar si los cambios funcionaban, encontrar qué parte había que corregir cuando aparecía un error y confirmar que lo anterior seguía funcionando.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Guía del Proyecto Inicial 2026-2 (Ciclo 3/4), enunciado del problema I Slot Machine (ICPC 2025) y materiales de clase (UML, colaboración entre objetos, división en ciclos, pruebas en Java, documentación, precondiciones e invariantes). La más útil fue el enunciado del problema.

Ciclo 4: refactorización y extensión

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos. Primero refactorizamos sin agregar nada nuevo, para comprobar con las pruebas anteriores que la base seguía funcionando. Después agregamos un grupo de tipos por mini-ciclo, y al final la parte visual, las pruebas y la documentación. Así cada paso se podía probar por separado.

Mini-ciclo	Contenido	Justificación
4.1	**Refactorizar:** `Wheel` y `Symbol` abstractas, `NormalWheel` y `NormalSymbol`, excepción `SlotMachineException`	Sin cambiar el comportamiento, comprobado con las pruebas de los ciclos 2 y 3 antes de extender.
4.2	Ruedas `lefty` y `rebel`, con `addWheel(type, pos)`	Cada tipo es una subclase que solo cambia lo que la hace diferente.
4.3	Símbolos `ephemeral` y `shy`, con `addSymbol(type, pos, color)`	Igual, con los métodos `onSpin()` y `onSelect()`.
4.4	Rueda propuesta `lazy`	Muestra que extender ya no cuesta: una clase y una línea en `createWheel`.
4.5	Usabilidad: marco de color por tipo de	Para que cada tipo se vea distinto.

	rueda y figura por tipo de símbolo	
4.6	Pruebas `SlotMachineC4Test` y `SlotMachineCC4Test`	Para cerrar el ciclo con pruebas.
4.7	Diseño en astah, pruebas de aceptación y documentación	Para que el diseño coincida con el código.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué? Terminado (4.1 a 4.7). Al cerrar el ciclo pasaban 83 de 83 pruebas, y las 41 pruebas de C4 y CC4 siguen pasando en el Cierre.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? Wilson Daniel Morales Sánchez: 14 horas. Samuel Mena Serrato: 14 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué? Agregar siete tipos de elementos donde cada tipo es una subclase pequeña que solo cambia lo que la hace diferente. Además, todo lo de los ciclos anteriores, incluido `solve`, sigue funcionando.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo? Que los símbolos se vieran distintos según su tipo, porque antes la rueda era solo un círculo pintado del color del símbolo. Lo solucionamos haciendo que cada símbolo sepa dibujarse con su propia figura, y que la rueda tenga un marco de color según su tipo. En el proceso encontramos que el fondo de la máquina tapaba las ruedas al cambiar de color, y lo corregimos volviendo a poner las ruedas encima.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar? Bien: correr las pruebas después de cada cambio, lo que mostró que la refactorización no dañó nada. A mejorar: pasar el diagrama a astah al terminar cada mini-ciclo y no al final.

7. ¿Cuál fue la práctica XP más útil? ¿Por qué? Refactorización junto con **Testing**: las pruebas de los ciclos anteriores permitieron cambiar la estructura interna de la máquina con seguridad.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Guía del Proyecto Inicial 2026-2 (Ciclo 4/4), enunciado del problema I Slot Machine (ICPC 2025) y materiales de clase sobre herencia y polimorfismo. Los más útiles fueron los materiales de herencia y polimorfismo, porque son la base de la refactorización.

Cierre (Ciclo 5): calidad con herramientas modernas

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Mini-ciclo	Contenido	Justificación
5.1	Pasar el proyecto a **VS Code** con Maven (`pom.xml`, `src/main/java`, `src/test/java`, `.vscode`) y unificar todas las pruebas en JUnit 5	VS Code reconoce el proyecto solo y deja correr las pruebas, con y sin cubrimiento, desde el panel Testing.
5.2	Tomar los **resultados iniciales** de JaCoCo y PMD sin cambiar nada	Para tener con qué comparar y decidir qué arreglar.
5.3	Crear **`SlotMachineC1Test`**, **`SlotMachineC3Test`** y **`SlotMachineAcceptanceTest`**	Lo pide la entrega, y además subieron el cubrimiento.
5.4	Corregir las **11 violaciones de prioridad alta** de PMD	Es la meta del análisis estático.
5.5	**Resultados finales**, informes, retrospectiva y zip de entrega	Para cerrar el proyecto.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué? Terminado (5.1 a 5.5). El proyecto abre y corre en VS Code. Pasan las **104 pruebas** (102 de unidad y 2 escenarios de aceptación). **JaCoCo:** 70,4 % inicial y **91,1 %** final (meta: más de 75 %). **PMD:** 11 violaciones de prioridad alta al inicio y **0** al final.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? Wilson Daniel Morales Sánchez: 11 horas. Samuel Mena Serrato: 11 horas.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué? Comprobar con herramientas la calidad del simulador: las pruebas cubren el 91,1 % del código y no queda ninguna regla de prioridad alta de PMD sin cumplir.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- **Pasar a VS Code:** dos clases de prueba usaban JUnit 4 y las demás JUnit 5, así que no corrían juntas. Las pasamos todas a JUnit 5.
- **PMD:** `Symbol` y `Wheel` tenían métodos vacíos "por defecto" (`onSpin`, `onSelect`, `checkDeletable`), y el constructor `SlotMachine(n)` llamaba métodos que se podían redefinir. Hicimos abstractos esos métodos, para que cada tipo diga explícitamente cómo reacciona, y declaramos `SlotMachine` como `final`.
- **Prueba de aceptación:** al hacer visible la máquina, los errores (por ejemplo, bloquear una rueda rebelde) muestran un mensaje que hay que cerrar con **Aceptar**. Lo dejamos así, porque es parte de lo que se quiere mostrar.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar? Bien: correr todas las pruebas después de cada corrección de PMD. A mejorar: correr JaCoCo y PMD desde el inicio de cada proyecto, no solo al cierre.

7. ¿Cuál fue la práctica XP más útil? ¿Por qué? Testing junto con la **refactorización**: las 104 pruebas nos permitieron corregir el código según PMD con la seguridad de que todo seguía funcionando igual.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil?

- Escuela Colombiana de Ingeniería. (2026). *Guía del Proyecto Inicial 2026-2, Cierre*. Desarrollo Orientado por Objetos.
- JaCoCo. (s.f.). *JaCoCo Java Code Coverage Library*. https://www.jacoco.org/jacoco/
- PMD. (s.f.). *Java Rules*. https://docs.pmd-code.org/latest/pmd_rules_java.html
- Microsoft. (s.f.). *Java in Visual Studio Code*. https://code.visualstudio.com/docs/languages/java
- Anthropic. (2026). *Claude*. Usado en el Cierre para migrar el proyecto a VS Code, escribir las pruebas nuevas y generar los informes. https://claude.ai

La más útil fue la documentación de reglas de PMD, porque explica qué significa cada violación y cómo corregirla.